

Name :- Mihir Mungara

POC Implementation

Lambda Function to Ingest Data (Airplanes.live API)

Navigation: Lambda > Functions > flight-data-poller

flight-data-poller

Throttle Copy ARN Actions

Function overview Info

Diagram Template

flight-data-poller

Layers (0)

+ Add trigger + Add destination

Export to Infrastructure Composer Download

Function ARN
arn:aws:lambda:ap-south-1:856021349192:function:flight-data-poller

Description
-

Last modified
3 hours ago

Code :-

```
import json
import boto3
import urllib.request
from datetime import datetime
from concurrent.futures import ThreadPoolExecutor, as_completed

s3 = boto3.client("s3")

BUCKET = "raw-flight-data-mm"

# Multiple points to cover all of India — each within the 250nm API limit
POLL_POINTS = [
    {"lat": 28.6139, "lon": 77.2090, "name": "delhi"},
    {"lat": 19.0760, "lon": 72.8777, "name": "mumbai"},
    {"lat": 13.0827, "lon": 80.2707, "name": "chennai"},
    {"lat": 12.9716, "lon": 77.5946, "name": "bangalore"},
    {"lat": 22.5726, "lon": 88.3639, "name": "kolkata"},
    {"lat": 23.0225, "lon": 72.5714, "name": "ahmedabad"},
    {"lat": 17.3850, "lon": 78.4867, "name": "hyderabad"},
]

def fetch_point(point):
    url = f"https://api.airplanes.live/v2/point/{point['lat']}/{point['lon']}/250"

    req = urllib.request.Request(
        url,
        headers={
            "User-Agent": "Mozilla/5.0"
        }
    )

    try:
        with urllib.request.urlopen(req, timeout=15) as response:
            data = json.loads(response.read().decode("utf-8"))
            return data.get("ac", [])
    except Exception as e:
        print(f"Failed for {point['name']}: {str(e)}")
        return []

def lambda_handler(event, context):
    all_aircraft = {} # dedupe by hex across overlapping city radii

    with ThreadPoolExecutor(max_workers=7) as executor:
        futures = {executor.submit(fetch_point, p): p for p in POLL_POINTS}
```

```

for future in as_completed(futures):
    aircraft_list = future.result()
    for ac in aircraft_list:
        hex_id = ac.get("hex")
        if hex_id:
            all_aircraft[hex_id] = ac

merged_data = {
    "ac": list(all_aircraft.values()),
    "total": len(all_aircraft),
    "now": int(datetime.utcnow().timestamp() * 1000)
}

now = datetime.utcnow()

key = (
    f"year={now.year}/"
    f"month={now.month:02d}/"
    f"day={now.day:02d}/"
    f"flight_data_{now.strftime('%Y%m%d_%H%M%S')}.json"
)

s3.put_object(
    Bucket=BUCKET,
    Key=key,
    Body=json.dumps(merged_data)
)

return {
    "statusCode": 200,
    "aircraft_count": len(all_aircraft),
    "points_polled": len(POLL_POINTS),
    "file": key
}

```

Amazon EventBridge Scheduled at rate of 1 Minute that Triggers Flight-Data-Poller Lambda Function :-

flight-data-poller
Edit
Enable
Delete
CloudFormation Template

Rule details
Info

Rule name flight-data-poller	Status ⏻ Disabled	Event bus name default	Type Scheduled Standard
Description -	Rule ARN arn:aws:events:ap-south-1:856021349192:rule/flight-data-poller	Event bus ARN arn:aws:events:ap-south-1:856021349192:event-bus/default	

Event schedule
Targets
Monitoring
Tags

Targets
Edit

Details	Target Name	Type	ARN	Input	Role
▼	flight-data-poller	Lambda function	arn:aws:lambda:ap-south-1:856021349192:function:flight-data-poller	Matched event	Amazon_EventBridge_Invoke_Lambda_1866940538
Input to target: Matched event Additional parameters: -- Dead-letter queue (DLQ): -					

Enabled Rules :-

flight-data-poller

EditDisableDeleteCloudFormation Template

Rule detailsInfo

Rule name
flight-data-poller

Description
-

Status
Enabled

Rule ARN
arn:aws:events:ap-south-1:856021349192:rule/flight-data-poller

Event bus name
default

Event bus ARN
arn:aws:events:ap-south-1:856021349192:event-bus/default

Type
Scheduled Standard

Event scheduleInfoEdit

Fixed rate of
1 minute

Raw Data Loaded into S3-Bucket :-

Amazon S3> Buckets> raw-flight-data-mm> year=2026/> month=06/> day=19/

day=19/

Copy S3 URI

ObjectsProperties

Objects (23)

Copy S3 URICopy URIDownloadOpen L?DeleteActionsCreate folderUpload

Find objects by prefixShow versions

	Name	Type	Last modified	Size	Storage class
☐	flight_data_20260619_085659.json	json	June 19, 2026, 14:27:00 (UTC+05:30)	124.2 KB	Standard
☐	flight_data_20260619_085759.json	json	June 19, 2026, 14:28:00 (UTC+05:30)	120.7 KB	Standard
☐	flight_data_20260619_085858.json	json	June 19, 2026, 14:28:59 (UTC+05:30)	119.1 KB	Standard
☐	flight_data_20260619_085958.json	json	June 19, 2026, 14:30:00 (UTC+05:30)	123.2 KB	Standard
☐	flight_data_20260619_090059.json	json	June 19, 2026, 14:31:00 (UTC+05:30)	121.4 KB	Standard
☐	flight_data_20260619_090158.json	json	June 19, 2026, 14:32:00 (UTC+05:30)	121.7 KB	Standard
☐	flight_data_20260619_090258.json	json	June 19, 2026, 14:33:00 (UTC+05:30)	122.1 KB	Standard

Event notifications (1)

EditDeleteCreate event notification

Send a notification when specific events occur in your bucket. Learn more

	Name	Event types	Filters	Destination type	Destination
☐	new-flight-file	All object create events	, json	SQS queue	raw-flight-data-events-mm

Amazon EventBridge

For additional capabilities, use Amazon EventBridge to build event-driven applications at scale using S3 event notifications. Learn more or see EventBridge pricing

Send notifications to Amazon EventBridge for all events in this bucket

Off

Edit

SQS Configured to Notify other Lambda Function which is responsible for Transformations :-

raw-flight-data-events-mm

[Edit](#) [Delete](#) [Purge](#) [Send and receive messages](#) [Start DLQ redrive](#)

Details [info](#)

Name raw-flight-data-events-mm	Type Standard	ARN arn:aws:sqs:ap-south-1:856021349192:raw-flight-data-events-mm
Encryption Amazon SQS key (SSE-SQS)	URL https://sqs.ap-south-1.amazonaws.com/856021349192/raw-flight-data-events-mm	Dead-letter queue -

[More](#)

[Queue policies](#) [Monitoring](#) [SNS subscriptions](#) [Lambda triggers](#) [EventBridge Pipes](#) [Dead-letter queue](#) [Tagging](#) [Encryption](#) [Dead-letter queue redrive tasks](#)

Lambda triggers (1) [info](#)

[View in Lambda](#) [Delete](#) [Configure Lambda function trigger](#)

UUID	ARN	Status	Last modified
2bc6cd0f-8fb8-4e0a-bdb3-3eb6acccb524e	arn:aws:lambda:ap-south-1:856021349192:function:flight-transformer	Enabled	17/6/2026, 2:37:30 pm

Second Lambda Function which is Responsible for Creating Silver Layer of Medallion Architecture :-

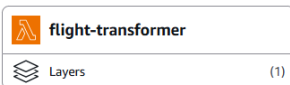
[Lambda](#) > [Functions](#) > flight-transformer

flight-transformer

[Throttle](#) [Copy ARN](#) [Actions](#)

Function overview [info](#)

[Diagram](#) [Template](#)



[Export to Infrastructure Composer](#) [Download](#)

Function ARN
[arn:aws:lambda:ap-south-1:856021349192:function:flight-transformer](#)

Description
-

Last modified
3 hours ago

Layers (1) [info](#)

Last fetched 19/6/2026, 5:42:15 pm [Refresh](#) [Edit](#)

Merge order	Name	Layer version	Compatible runtimes	Compatible architectures	Version ARN
1	AWSSDKPandas-Python312	27	python3.12	x86_64	arn:aws:lambda:ap-south-1:336392948345:layer:AWSSDKPandas-Python312:27

Code :-

```
import json
import boto3
import pandas as pd
import io
from datetime import datetime, timezone
from urllib.parse import unquote_plus

s3 = boto3.client("s3")
SILVER_BUCKET = "flight-silver-bucket-mm"

def lambda_handler(event, context):
    for record in event["Records"]:
        body = json.loads(record["body"])
        s3_info = json.loads(body["Message"]) if "Message" in body else body

        for s3_record in s3_info.get("Records", []):
            bucket = s3_record["s3"]["bucket"]["name"]
            key = unquote_plus(s3_record["s3"]["object"]["key"])

            obj = s3.get_object(Bucket=bucket, Key=key)
```

```

data = json.loads(obj["Body"].read().decode("utf-8"))

aircraft_list = data.get("ac", [])
snapshot_ts = datetime.fromtimestamp(
    data.get("now", 0) / 1000, tz=timezone.utc
).isoformat()

rows = [extract_fields(ac, snapshot_ts) for ac in aircraft_list]

if not rows:
    continue

df = pd.DataFrame(rows)

df["alt_baro"] = df["alt_baro"].replace("ground", 0)

numeric_cols = [
    "alt_baro",
    "alt_geom",
    "baro_rate",
    "geom_rate",
    "gs",
    "ias",
    "tas",
    "mach",
    "latitude",
    "longitude"
]

for col in numeric_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")

# Add derived columns right here in Lambda
df["flight_phase"] = df["alt_baro"].apply(classify_phase)
df["vertical_trend"] = df["baro_rate"].apply(classify_vertical)
df["speed_category"] = df["gs"].apply(classify_speed)
df["is_heavy"] = df["category"].isin(["A5"])
df["grid_lat"] = df["latitude"].apply(
    lambda x: int(x) if pd.notna(x) else None
)
df["grid_lon"] = df["longitude"].apply(
    lambda x: int(x) if pd.notna(x) else None
)
df["grid_cell"] = df["grid_lat"].astype(str) + "_" + \
    df["grid_lon"].astype(str)

# Write parquet
buf = io.BytesIO()
df.to_parquet(buf, engine="pyarrow", index=False)
buf.seek(0)

now = datetime.utcnow()
out_key = (
    f"year={now.year}/month={now.month:02d}/"
    f"day={now.day:02d}/"
    f"{key.split('/')[0].replace('.json', '.parquet')}"
)

s3.put_object(Bucket=SILVER_BUCKET, Key=out_key, Body=buf.read())

return {"statusCode": 200}

```

```

def extract_fields(ac, snapshot_ts):
    return {
        # --- Identity ---
        "hex": ac.get("hex"),
        "flight": (ac.get("flight") or "").strip() or None,
        "registration": ac.get("r"),
        "aircraft_type": ac.get("t"),
        "aircraft_desc": ac.get("desc"),
        "category": ac.get("category"),
    }

```

```

"adsb_type": ac.get("type"),

# --- Position ---
"latitude": ac.get("lat"),
"longitude": ac.get("lon"),
"alt_baro": ac.get("alt_baro"),
"alt_geom": ac.get("alt_geom"),

# --- Speed & Movement ---
"gs": ac.get("gs"), # ground speed (knots)
"ias": ac.get("ias"), # indicated airspeed
"tas": ac.get("tas"), # true airspeed
"mach": ac.get("mach"),
"track": ac.get("track"), # heading degrees
"baro_rate": ac.get("baro_rate"), # ft/min climb/descent
"geom_rate": ac.get("geom_rate"),
"roll": ac.get("roll"),

# --- Atmospheric ---
"wind_dir": ac.get("wd"),
"wind_speed": ac.get("ws"),
"oat": ac.get("oat"), # outside air temp °C
"tat": ac.get("tat"), # total air temp °C

# --- Navigation ---
"squawk": ac.get("squawk"),
"emergency": ac.get("emergency"),
"nav_altitude": ac.get("nav_altitude_mcp"),
"nav_heading": ac.get("nav_heading"),
"nav_qnh": ac.get("nav_qnh"),

# --- Signal Quality ---
"nic": ac.get("nic"), # navigation integrity
"rsi": ac.get("rsi"), # signal strength
"messages": ac.get("messages"), # total ADS-B messages
"seen": ac.get("seen"), # seconds since last msg
"seen_pos": ac.get("seen_pos"), # seconds since last pos

# --- Distance from receiver ---
"dst": ac.get("dst"), # distance (nm)
"dir": ac.get("dir"), # direction from receiver

# --- Snapshot metadata ---
"snapshot_ts": snapshot_ts,
"processed_time": datetime.utcnow().isoformat(),
}

# --- Derived column helpers ---
def classify_phase(alt):
    # Handle string "ground" status and any other non-numeric values
    if alt is None:
        return "Unknown"
    if isinstance(alt, str):
        if alt.lower() == "ground":
            return "Ground/Taxi"
    try:
        alt = float(alt)
    except (ValueError, TypeError):
        return "Unknown"
    if alt < 1000:
        return "Ground/Taxi"
    elif alt < 10000:
        return "Climb/Descent"
    else:
        return "Cruise"

def classify_vertical(rate):
    if rate is None:
        return "Unknown"

    try:
        rate = float(rate)

```

```
if gs < 150:
    return "Slow (<150 kts)"
elif gs < 350:
    return "Subsonic (150-350)"
else:
    return "High Speed (350+)"
```

Amazon S3

Buckets

flight-silver-bucket-mm

year=2026/

month=06/

day=19/

day=19/

Copy S3 URI

Objects

Properties

Objects (23)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Find objects by prefix

Show versions

1

	Name	Type	Last modified	Size	Storage class
☐	flight_data_20260619_085659.parquet	parquet	June 19, 2026, 14:27:02 (UTC+05:30)	49.9 KB	Standard
☐	flight_data_20260619_085759.parquet	parquet	June 19, 2026, 14:28:01 (UTC+05:30)	49.6 KB	Standard
☐	flight_data_20260619_085858.parquet	parquet	June 19, 2026, 14:29:01 (UTC+05:30)	48.8 KB	Standard
☐	flight_data_20260619_085958.parquet	parquet	June 19, 2026, 14:30:01 (UTC+05:30)	49.5 KB	Standard
☐	flight_data_20260619_090059.parquet	parquet	June 19, 2026, 14:31:02 (UTC+05:30)	49.3 KB	Standard
☐	flight_data_20260619_090158.parquet	parquet	June 19, 2026, 14:32:01 (UTC+05:30)	49.5 KB	Standard
☐	flight_data_20260619_090258.parquet	parquet	June 19, 2026, 14:33:01 (UTC+05:30)	49.7 KB	Standard
☐	flight_data_20260619_090358.parquet	parquet	June 19, 2026, 14:34:01 (UTC+05:30)	49.7 KB	Standard
☐	flight_data_20260619_090459.parquet	parquet	June 19, 2026, 14:35:02 (UTC+05:30)	50.2 KB	Standard
☐	flight_data_20260619_090558.parquet	parquet	June 19, 2026, 14:36:01 (UTC+05:30)	49.9 KB	Standard

Event notifications (1)

Edit

Delete

Create event notification

Send a notification when specific events occur in your bucket. [Learn more](#)

	Name	Event types	Filters	Destination type	Destination
☐	snowflake_notification	All object create events	-	SQS queue	arn:aws:sqs:ap-south-1:877013879214:sf-snowpipe-AIDA4YMQZZGXJZ2K5WKEQ-JYQgz0hePon7wP1Yb0xfw

Snowflake Code :-

```
-- used appropriate role
USE ROLE ACCOUNTADMIN;

-- created database and warehouse
CREATE DATABASE FLIGHT_DB;
CREATE SCHEMA FLIGHT_DB.STAGING;

CREATE WAREHOUSE FLIGHT_WH
WITH
WAREHOUSE_SIZE='XSMALL'
AUTO_SUSPEND=60
AUTO_RESUME=TRUE;

-- created storage integration object
CREATE OR REPLACE STORAGE INTEGRATION flight_s3_int
TYPE = EXTERNAL_STAGE
STORAGE_PROVIDER = S3
ENABLED = TRUE
STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::856021349192:role/SnowflakeS3Role'
STORAGE_ALLOWED_LOCATIONS = (
    's3://flight-silver-bucket-mm/'
);

DESC INTEGRATION flight_s3_int;

USE WAREHOUSE FLIGHT_WH;
USE DATABASE FLIGHT_DB;
USE SCHEMA STAGING;

-- created file format
CREATE OR REPLACE FILE FORMAT parquet_ff
TYPE = PARQUET;

-- created stage
CREATE OR REPLACE STAGE silver_stage
URL='s3://flight-silver-bucket-mm/'
STORAGE_INTEGRATION=flight_s3_int
FILE_FORMAT=parquet_ff;

list @silver_stage;

SELECT *
FROM @silver_stage
LIMIT 5;

-- created tables
CREATE OR REPLACE TABLE stg_flights (
    -- Identity
    hex        STRING,
    flight     STRING,
    registration STRING,
    aircraft_type STRING,
    aircraft_desc STRING,
    category   STRING,
    adsb_type  STRING,

    -- Position
    latitude   FLOAT,
    longitude  FLOAT,
    alt_baro   NUMBER,
    alt_geom   NUMBER,

    -- Speed & Movement
    gs         FLOAT,
    ias        FLOAT,
    tas        FLOAT,
    mach       FLOAT,
    track      FLOAT,
```



```

baro_rate    FLOAT,
geom_rate    FLOAT,
roll         FLOAT,

-- Atmospheric
wind_dir     FLOAT,
wind_speed   FLOAT,
oat          FLOAT,
tat          FLOAT,

-- Navigation
squawk       STRING,
emergency    STRING,
nav_altitude FLOAT,
nav_heading  FLOAT,
nav_qnh      FLOAT,

-- Signal Quality
nic          NUMBER,
rssi         FLOAT,
messages     NUMBER,
seen         FLOAT,
seen_pos     FLOAT,

-- Distance from receiver
dst          FLOAT,
dir          FLOAT,

-- Derived
flight_phase STRING,
vertical_trend STRING,
speed_category STRING,
is_heavy     BOOLEAN,
grid_lat     NUMBER,
grid_lon     NUMBER,
grid_cell    STRING,

-- Timestamps
snapshot_ts   TIMESTAMP_NTZ,
processed_time TIMESTAMP_NTZ
);

```

```

-- created snowpipe for automatic data ingestion
CREATE OR REPLACE PIPE flight_pipe
AUTO_INGEST = TRUE
AS
COPY INTO stg_flights
FROM @silver_stage
MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE;

```

```

SELECT COUNT(*)
FROM stg_flights;

```

```

SELECT *
FROM stg_flights
LIMIT 100;

```

```

DESC PIPE flight_pipe;
SHOW PIPES;

```

```

ALTER PIPE flight_pipe SET PIPE_EXECUTION_PAUSED = TRUE;
ALTER PIPE flight_pipe SET PIPE_EXECUTION_PAUSED = FALSE;

```

```

SELECT SYSTEM$PIPE_STATUS('flight_pipe');

```

```

DROP PIPE IF EXISTS flight_pipe;
TRUNCATE TABLE stg_flights;

```

```
-- airports.csv stage
CREATE OR REPLACE STORAGE INTEGRATION airport_s3_int
TYPE = EXTERNAL_STAGE
STORAGE_PROVIDER = S3
ENABLED = TRUE
STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::856021349192:role/s3_full_access_role'
STORAGE_ALLOWED_LOCATIONS = (
    's3://extra-resources-mm/'
);
```

```
CREATE OR REPLACE STAGE AIRPORT_STAGE
URL='s3://extra-resources-mm/'
STORAGE_INTEGRATION = airport_s3_int
FILE_FORMAT = MY_CSV_FORMAT;
```

```
DESC INTEGRATION airport_s3_int;
```

```
list @AIRPORT_STAGE;
```

```
-- created another required tables
CREATE OR REPLACE TABLE RAW_AIRPORTS (
    ID NUMBER,
    IDENT STRING,
    TYPE STRING,
    NAME STRING,
    LATITUDE_DEG FLOAT,
    LONGITUDE_DEG FLOAT,
    ELEVATION_FT NUMBER,
    CONTINENT STRING,
    ISO_COUNTRY STRING,
    ISO_REGION STRING,
    MUNICIPALITY STRING,
    SCHEDULED_SERVICE STRING,
    ICAO_CODE STRING,
    IATA_CODE STRING,
    GPS_CODE STRING,
    LOCAL_CODE STRING,
    HOME_LINK STRING,
    WIKIPEDIA_LINK STRING,
    KEYWORDS STRING
);
```

```
COPY INTO RAW_AIRPORTS
FROM @AIRPORT_STAGE/airports.csv
FILE_FORMAT = (
    TYPE = CSV
    SKIP_HEADER = 1
    FIELD_OPTIONALLY_ENCLOSED_BY = ''
);
```

```
SELECT *
FROM RAW_AIRPORTS
LIMIT 10;
```

```
CREATE OR REPLACE TABLE DIM_AIRPORTS AS
SELECT
    IDENT,
    IATA_CODE,
    ICAO_CODE,
    NAME,
    MUNICIPALITY,
    LATITUDE_DEG,
    LONGITUDE_DEG,
    ELEVATION_FT,
    TYPE,
    ISO_REGION
FROM RAW_AIRPORTS
WHERE ISO_COUNTRY = 'IN';

SELECT COUNT(*)
```

```
FROM DIM_AIRPORTS;
```

```
-- regions
```

```
CREATE OR REPLACE TABLE RAW_REGIONS (  
  STATE_NAME STRING,  
  MIN_LAT FLOAT,  
  MAX_LAT FLOAT,  
  MIN_LON FLOAT,  
  MAX_LON FLOAT  
);
```

```
COPY INTO RAW_REGIONS  
FROM @AIRPORT_STAGE/dim_regions.csv  
FILE_FORMAT = (  
  TYPE = CSV  
  SKIP_HEADER = 1  
  FIELD_OPTIONALLY_ENCLOSED_BY = ''''  
);
```

```
SELECT * FROM RAW_REGIONS;
```

```
ALTER TABLE RAW_REGIONS  
RENAME TO DIM_REGIONS;
```

```
-- Check dimensions populated correctly
```

```
SELECT  
  indian_state,  
  nearest_airport_city,  
  aircraft_family,  
  COUNT(*) AS flights  
FROM flight_db.staging_gold.fct_flights  
GROUP BY 1,2,3  
ORDER BY 4 DESC;
```

```
select * from flight_db.staging_gold.fct_flights ;
```

```
SELECT *  
FROM FLIGHT_DB.STAGING_GOLD.DIM_AIRCRAFT  
LIMIT 20;
```

```
SELECT DISTINCT aircraft_type  
FROM FLIGHT_DB.STAGING.STG_FLIGHTS  
LIMIT 20;
```

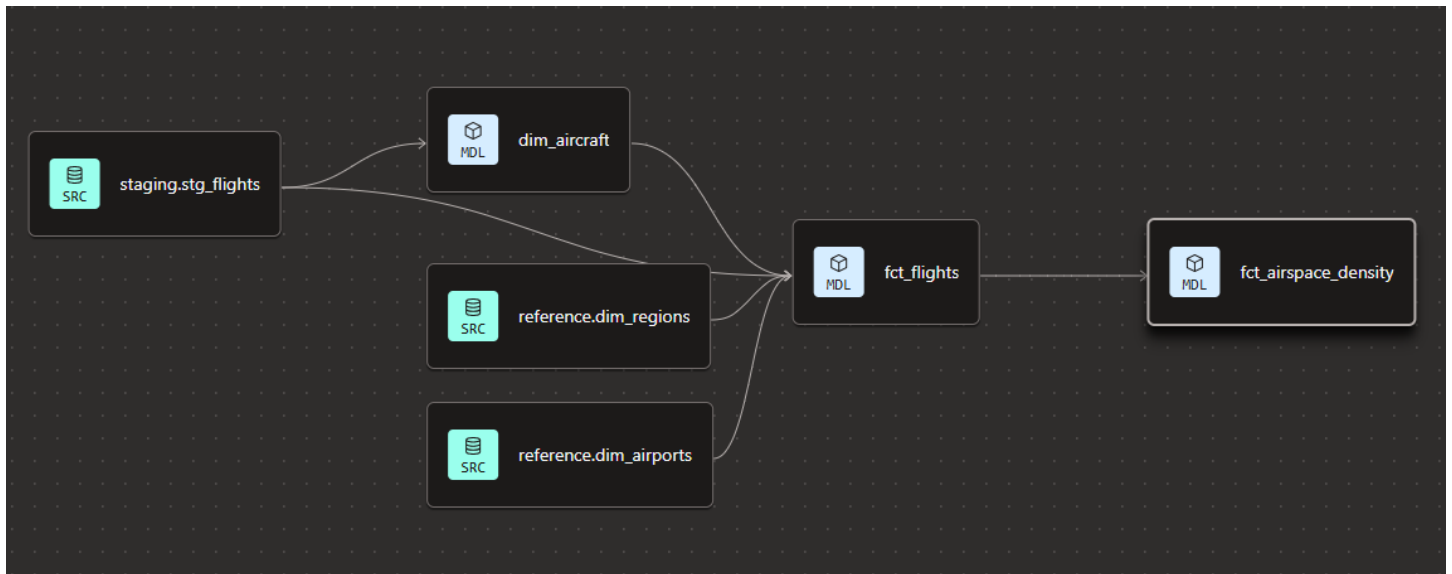
```
desc table FLIGHT_DB.STAGING_GOLD.DIM_AIRCRAFT;  
desc table FLIGHT_DB.STAGING_GOLD.FCT_AIRSPACE_DENSITY;  
desc table FLIGHT_DB.STAGING_GOLD.FCT_FLIGHTS;  
desc table FLIGHT_DB.STAGING_GOLD.FCT_ROLLING_STATS;
```

```
SELECT * FROM FLIGHT_DB.STAGING_GOLD.DIM_AIRCRAFT;  
SELECT * FROM FLIGHT_DB.STAGING_GOLD.FCT_AIRSPACE_DENSITY;  
SELECT * FROM FLIGHT_DB.STAGING_GOLD.FCT_FLIGHTS;  
SELECT * FROM FLIGHT_DB.STAGING_GOLD.FCT_ROLLING_STATS;
```

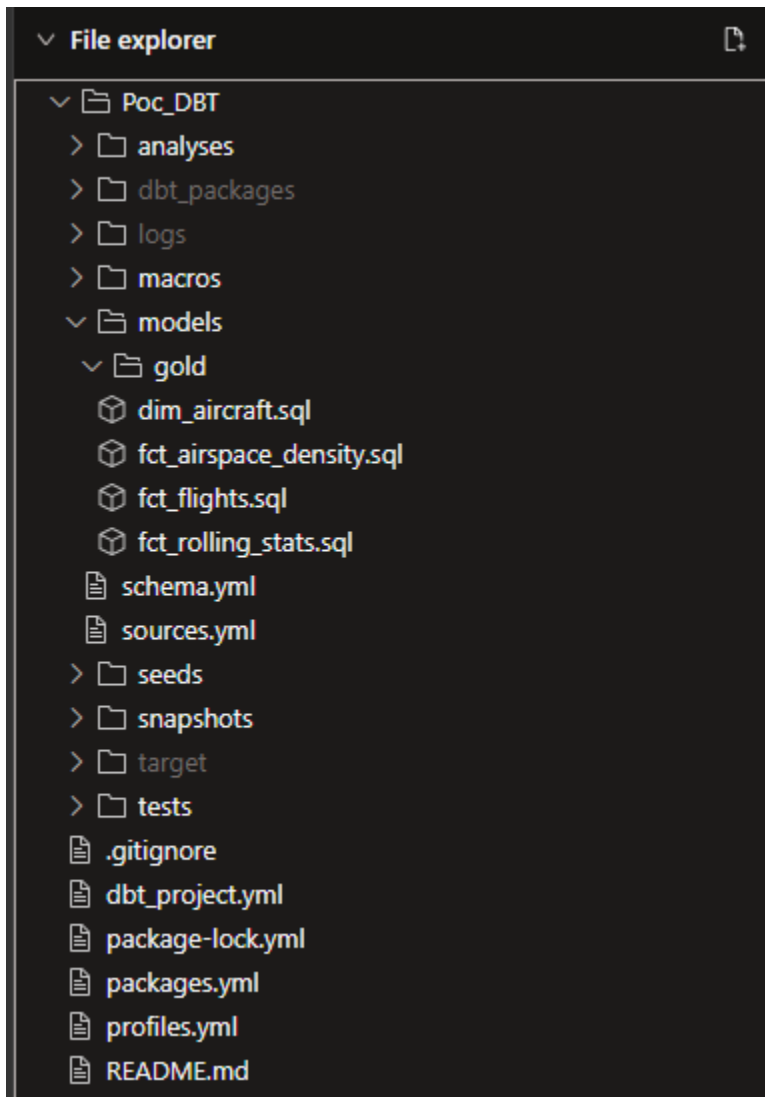
```
drop table FLIGHT_DB.STAGING_GOLD.DIM_AIRCRAFT;  
drop table FLIGHT_DB.STAGING_GOLD.FCT_AIRSPACE_DENSITY;  
drop table FLIGHT_DB.STAGING_GOLD.FCT_FLIGHTS;  
drop table FLIGHT_DB.STAGING_GOLD.FCT_ROLLING_STATS;
```

```
truncate table FLIGHT_DB.STAGING.STG_FLIGHTS;
```

DBT Part:-



DBT File Structure :-



Dbt_project.yml :-

```
Poc_DBT > dbt_project.yml

DbtProject (dbt_project-latest-fusion.json)
1  name: flight_telemetry
2  version: '1.0.0'
3  config-version: 2
4
5  profile: default
6
7  model-paths: ["models"]
8  seed-paths:  ["seeds"]
9  test-paths:  ["tests"]
10
11 models:
12   flight_telemetry:
13     gold:
14       +schema: gold
15       +database: FLIGHT_DB
```

Schema.yml Code :-

```
# models/schema.yml
version: 2

models:
  - name: fct_flights
    description: "Cleaned, enriched flight records — base fact table"

    columns:
      - name: hex
        tests:
          - not_null

      - name: snapshot_ts
        tests:
          - not_null

      - name: latitude
        tests:
          - not_null

      - name: longitude
        tests:
          - not_null
```

```
- name: flight_phase
tests:
  - accepted_values:
      arguments:
        values:
          - Ground/Taxi
          - Climb/Descent
          - Cruise
          - Unknown

- name: vertical_trend
tests:
  - accepted_values:
      arguments:
        values:
          - Climbing
          - Descending
          - Level
          - Unknown

- name: fct_airspace_density

columns:
  - name: aircraft_count
    tests:
      - not_null

- name: grid_cell
tests:
  - not_null
```

Sources.yml code :-

```
sources:
  - name: staging
    database: FLIGHT_DB
    schema: STAGING
    tables:
      - name: stg_flights
        description: "Bronze→Silver ingested flight records from Airplanes.live"
        config:
```

```
loaded_at_field: snapshot_ts
freshness:
  warn_after:
    count: 10
    period: minute
  error_after:
    count: 20
    period: minute
columns:
  - name: hex
    tests: [not_null]
  - name: snapshot_ts
    tests: [not_null]
  - name: latitude
    tests: [not_null]
  - name: longitude
    tests: [not_null]

- name: reference
  database: FLIGHT_DB
  schema: STAGING
  tables:
    - name: dim_airports
    - name: dim_regions
```

Model1 (dim_aircraft) :-

```
-- models/gold/dim_aircraft.sql
{{
  config(
    materialized = 'table'
  )
}}
```

```
SELECT DISTINCT
  aircraft_type,
```

```
-- Manufacturer: purely from first 1-2 chars, no hardcoding
CASE
  WHEN LEFT(aircraft_type, 2) = 'DH' THEN 'De Havilland'
  WHEN LEFT(aircraft_type, 2) = 'AT' THEN 'ATR'
  WHEN LEFT(aircraft_type, 1) = 'A' THEN 'Airbus'
```

```

WHEN LEFT(aircraft_type, 1) = 'B' THEN 'Boeing'
WHEN LEFT(aircraft_type, 1) = 'E' THEN 'Embraer'
WHEN LEFT(aircraft_type, 1) = 'F' THEN 'Dassault'
WHEN LEFT(aircraft_type, 1) = 'C' THEN 'Cessna'
WHEN LEFT(aircraft_type, 1) = 'P' THEN 'Piper'
ELSE 'Unknown'
END AS manufacturer,

-- Family: just first 3 chars groups same base model
-- e.g. A320, A20N → "A32" / "A20" — no need to list every variant
LEFT(aircraft_type, 3) AS family_code,

-- Category label: comes directly from API, no hardcoding needed
CASE
  WHEN category = 'A1' THEN 'Light'
  WHEN category = 'A2' THEN 'Small'
  WHEN category = 'A3' THEN 'Medium'
  WHEN category = 'A4' THEN 'Large'
  WHEN category = 'A5' THEN 'Heavy'
  WHEN category = 'A6' THEN 'High Vortex'
  WHEN category = 'A7' THEN 'Rotorcraft'
  WHEN category = 'B1' THEN 'Glider'
  WHEN category = 'B2' THEN 'Lighter than air'
  WHEN category = 'B3' THEN 'Parachutist'
  WHEN category = 'B4' THEN 'Ultralight'
  WHEN category = 'B6' THEN 'UAV'
  WHEN category = 'C1' THEN 'Surface Vehicle'
  ELSE 'Unknown'
END AS category_label,

-- Wide body: purely from category — A5 (Heavy) are widebodies
-- No need to list every aircraft type
CASE
  WHEN category = 'A5' THEN TRUE
  ELSE FALSE
END AS wide_body,

-- Engine type: from category, no hardcoding aircraft types
CASE
  WHEN category = 'A7' THEN 'Rotorcraft'
  WHEN category IN ('A1', 'A2') THEN 'Piston/Turboprop'

```



```

    WHEN category IN ('A3','A4','A5') THEN 'Turbofan'
    ELSE 'Unknown'
END AS engine_type,

-- These come directly from API — no derivation needed
-- Just aggregate stats from your actual data
COUNT(*)           AS total_observations,
COUNT(DISTINCT hex) AS unique_aircraft_seen,
COUNT(DISTINCT flight) AS unique_flights_seen,
ROUND(AVG(alt_baro), 0) AS avg_cruise_altitude_ft,
ROUND(AVG(gs), 1)      AS avg_cruise_speed_kts,
ROUND(AVG(mach), 3)    AS avg_mach_number,
MIN(snapshot_ts)      AS first_seen,
MAX(snapshot_ts)      AS last_seen

FROM {{ source('staging', 'stg_flights') }}
WHERE aircraft_type IS NOT NULL

GROUP BY
    aircraft_type,
    manufacturer,
    family_code,
    category_label,
    wide_body,
    engine_type

```

Model2 (fct_airspace_density) :-

```

-- models/gold/fct_airspace_density.sql
{{
    config(
        materialized = 'incremental',
        unique_key    = ['snapshot_minute', 'grid_cell', 'flight_phase']
    )
}}

SELECT
    snapshot_minute,
    grid_cell,
    grid_lat,
    grid_lon,

```

```

flight_phase,

COUNT(*)                AS aircraft_count,
COUNT(DISTINCT aircraft_type) AS unique_aircraft_types,
COUNT(DISTINCT CASE WHEN is_heavy THEN hex END) AS heavy_count,
COUNT(DISTINCT CASE WHEN is_emergency THEN hex END) AS emergency_count,

ROUND(AVG(alt_baro), 0)    AS avg_altitude_ft,
ROUND(AVG(gs), 1)         AS avg_ground_speed,
ROUND(AVG(mach), 3)       AS avg_mach,
ROUND(MAX(gs), 1)        AS max_ground_speed,

ROUND(AVG(oat), 1)        AS avg_oat,
ROUND(AVG(wind_speed), 1) AS avg_wind_speed,
ROUND(AVG(rssi), 1)       AS avg_signal_rssi,

SUM(CASE WHEN vertical_trend = 'Climbing'
      THEN 1 ELSE 0 END) AS climbing_count,
SUM(CASE WHEN vertical_trend = 'Descending'
      THEN 1 ELSE 0 END) AS descending_count,
SUM(CASE WHEN vertical_trend = 'Level'
      THEN 1 ELSE 0 END) AS level_count

FROM {{ ref('fct_flights') }}

{% if is_incremental() %}
WHERE snapshot_minute > (SELECT MAX(snapshot_minute) FROM {{ this }})
{% endif %}

GROUP BY 1,2,3,4,5

```

Model3 (fct_flights) :-

```

{{
  config(
    materialized      = 'incremental',
    unique_key        = ['hex', 'snapshot_ts'],
    on_schema_change  = 'sync_all_columns'
  )
}}

```

```
SELECT
```

```
-- Identity
```

```
f.hex,
```

```
f.flight,
```

```
f.registration,
```

```
f.aircraft_type,
```

```
f.aircraft_desc,
```

```
f.category,
```

```
-- Position
```

```
f.latitude,
```

```
f.longitude,
```

```
f.alt_baro,
```

```
f.alt_geom,
```

```
-- Speed & Movement
```

```
f.gs,
```

```
f.ias,
```

```
f.tas,
```

```
f.mach,
```

```
f.track,
```

```
f.baro_rate,
```

```
f.geom_rate,
```

```
f.roll,
```

```
-- Atmospheric
```

```
f.wind_dir,
```

```
f.wind_speed,
```

```
f.oat,
```

```
f.tat,
```

```
(f.tat - f.oat)          AS temp_rise,
```

```
-- Navigation
```

```
f.squawk,
```

```
f.emergency,
```

```
f.nav_altitude,
```

```
f.nav_heading,
```

```
f.nav_qnh,
```

```
-- Signal
```

```
f.nic,
```

```

f.rssi,
f.messages,
f.seen,
f.dst,

-- Pre-computed derived (from Lambda)
f.flight_phase,
f.vertical_trend,
f.speed_category,
f.is_heavy,
f.grid_cell,
f.grid_lat,
f.grid_lon,

-- Extra derived in dbt
(f.alt_geom - f.alt_baro)          AS geom_baro_diff,
CASE
  WHEN f.squawk IN ('7500','7600','7700')
  THEN TRUE ELSE FALSE
END                                AS is_special_squawk,
CASE
  WHEN f.emergency != 'none'
  AND f.emergency IS NOT NULL
  THEN TRUE ELSE FALSE
END                                AS is_emergency,
EXTRACT(HOUR FROM f.snapshot_ts)   AS hour_of_day,
DAYNAME(f.snapshot_ts)            AS day_of_week,
DATE_TRUNC('minute', f.snapshot_ts) AS snapshot_minute,
f.snapshot_ts,
f.processed_time,

-- — dim_aircraft enrichment —————
da.manufacturer,
da.family_code          AS aircraft_family,
da.category_label,
da.wide_body,
da.engine_type,

-- — dim_regions enrichment —————
dr.state_name          AS indian_state,

```

```

-- — dim_airports enrichment —————
ap.iata_code          AS nearest_airport_iata,
ap.name               AS nearest_airport_name,
ap.municipality       AS nearest_airport_city,
ap.type               AS nearest_airport_type,
ap.elevation_ft       AS nearest_airport_elevation_ft,
ROUND(
  SQRT(
    POWER(f.latitude - ap.latitude_deg, 2) +
    POWER(f.longitude - ap.longitude_deg, 2)
  ) * 111.32
, 1)                  AS dist_to_nearest_airport_km

FROM {{ source('staging', 'stg_flights') }} f

-- — dim_aircraft —————
LEFT JOIN {{ ref('dim_aircraft') }} da
  ON f.aircraft_type = da.aircraft_type

-- — dim_regions —————
LEFT JOIN {{ source('reference', 'dim_regions') }} dr
  ON f.latitude BETWEEN dr.min_lat AND dr.max_lat
  AND f.longitude BETWEEN dr.min_lon AND dr.max_lon

-- — dim_airports (nearest airport per flight) ———
LEFT JOIN {{ source('reference', 'dim_airports') }} ap
  ON ap.latitude_deg BETWEEN f.latitude - 2 AND f.latitude + 2
  AND ap.longitude_deg BETWEEN f.longitude - 2 AND f.longitude + 2

WHERE f.latitude IS NOT NULL
  AND f.longitude IS NOT NULL
  AND f.hex IS NOT NULL

{% if is_incremental() %}
  AND f.snapshot_ts >
    (
      SELECT COALESCE(MAX(snapshot_ts), '1900-01-01'::TIMESTAMP)
      FROM {{ this }}
    )
{% endif %}

```

```
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY f.hex, f.snapshot_ts
  ORDER BY
    COALESCE(
      POWER(f.latitude - ap.latitude_deg, 2) +
      POWER(f.longitude - ap.longitude_deg, 2),
      99999999
    ) ASC
) = 1
```

Model4 (fct_rolling_stats) :-

```
-- models/gold/fct_rolling_stats.sql
{{
  config(materialized = 'table')
}}

SELECT
  hex,
  flight,
  aircraft_type,
  snapshot_ts,
  alt_baro,
  gs,
  mach,
  baro_rate,
  vertical_trend,

  -- 5-snapshot rolling average altitude per aircraft
  AVG(alt_baro) OVER (
    PARTITION BY hex
    ORDER BY snapshot_ts
    ROWS BETWEEN 4 PRECEDING AND CURRENT ROW
  ) AS rolling_avg_altitude,

  -- 5-snapshot rolling average speed per aircraft
  AVG(gs) OVER (
    PARTITION BY hex
    ORDER BY snapshot_ts
    ROWS BETWEEN 4 PRECEDING AND CURRENT ROW
  ) AS rolling_avg_speed,

  -- Max altitude this aircraft has reached
```

```

MAX(alt_baro) OVER (
  PARTITION BY hex
) AS session_max_altitude,

-- Rank by speed within each snapshot
RANK() OVER (
  PARTITION BY snapshot_minute
  ORDER BY gs DESC
) AS speed_rank_in_snapshot

FROM {{ ref('fct_flights') }}

```

DBT Job Scheduled for 5 Minutes :-

Orchestration / Environments / PRODUCTION **PROD** / flight_telemetry_prod

flight_telemetry_prod [Edit](#)

Job settings

Job name

flight_telemetry_prod

Consider choosing a name that's easily understood by your teammates.

[+ Add description](#)

Environment

PRODUCTION

Execution settings

Commands ⓘ

☒ Run source freshness

Enables dbt source freshness as the first step of this job, without breaking subsequent steps. [See the docs for more info.](#)

- dbt source freshness
- dbt build

Triggers

Configure when this job should run.

Schedule

☒ Run on schedule

Triggers a run when the schedule is active.

Timing

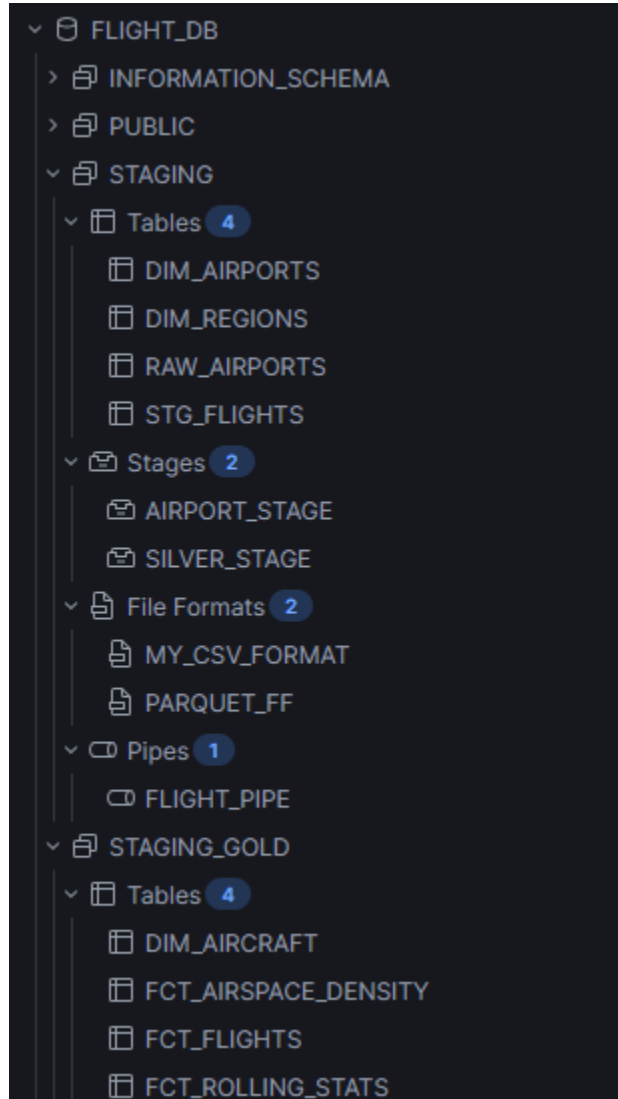
Cron schedule

Custom cron schedule (UTC) ⓘ

Every 2 minutes

Using cron syntax, you can specify the minute, hour, day of the month, month, and day of the week, allowing you to set up complex schedules like running a job on the first Monday of each month.

Snowflake Database after DBT Build :-



Dashboard-1 :-

 Real-Time India Flight Telemetry
Live ADS-B Data | Powered by Airplanes.live

149

Total Active Flights

• Live | Updates every 5 min

376.18 kts

Average Speed

• Ground Speed | Knots (kts)

24,069

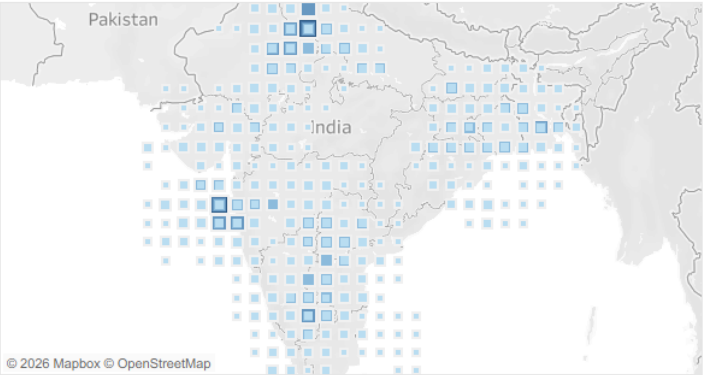
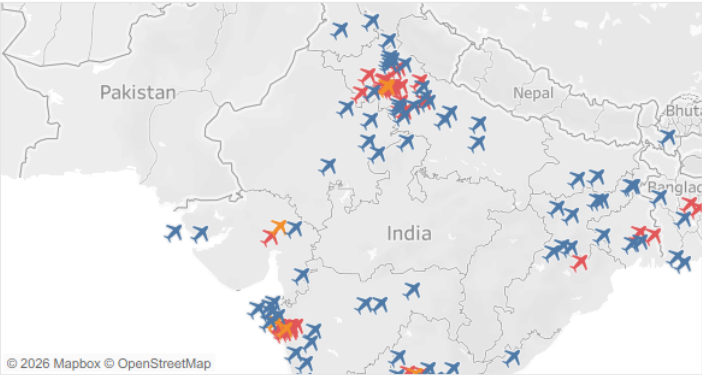
Average Altitude

• Barometric Altitude | Feet (ft)

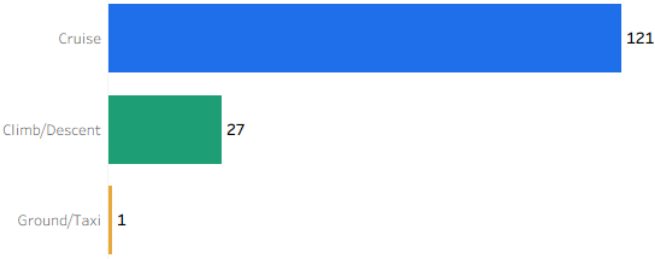
25

Heavy Aircraft Count

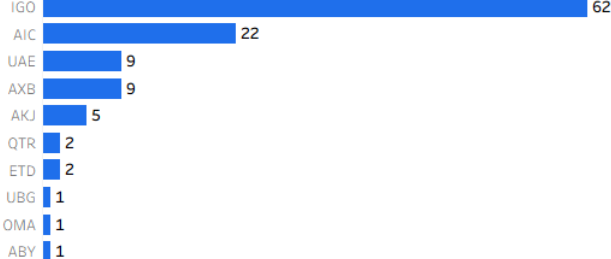
• ADS-B Category A5 Aircraft



Flight Phase



Airline



Dashboard-2 :-

🇮🇳 India Airspace Operations Center
Last Updated: 2026-06-23 09:35:47

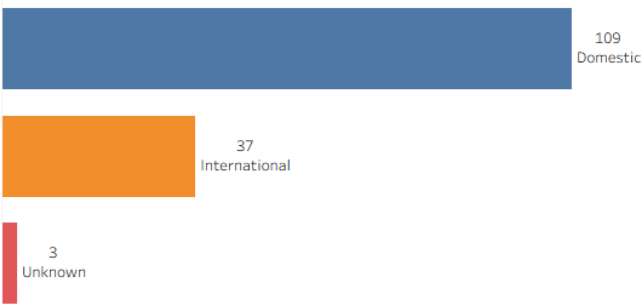
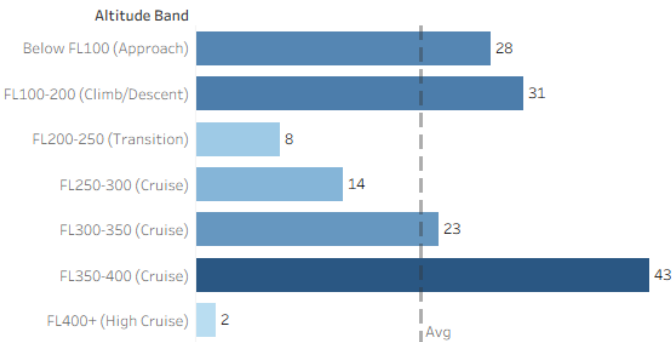
90
● Cruise Aircraft

22
● Climbing Aircraft

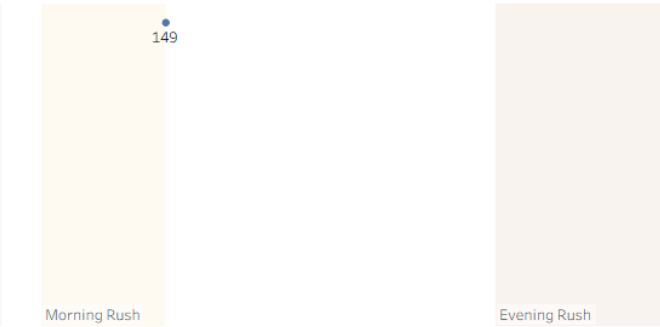
37
● Descending Aircraft

1
● Ground Aircraft

0
● Special Squawk Alerts



Nearest Air..	Nearest Air..	Approaching ↓	Departing ↑	On Ground ●	Total →
BLR	Bengaluru	3	0	0	4
BOM	Mumbai	1	0	1	1
DEL	New Delhi	0	1	0	1
HYD	Hyderabad	1	2	0	5
IXA	Agartala	1	0	0	2
PNQ	Pune	0	0	0	1
VTZ	Visakhapat..	0	0	0	1



Architecture :-

REAL-TIME INDIA FLIGHT TELEMETRY DATA ARCHITECTURE

Airplanes.live → AWS → Snowflake → dbt → Tableau

